

Sentiment-Analyse von Produktbewertungen als Cloud-/Big-Data-Prototyp

Lösungsdokumentation zu Projekt 6

Modul Cloud Computing & Big Data

DHBW Lörrach · Data Science und Künstliche Intelligenz

Julian Kenk, Mario Raach, René Schlesier

Stand: Juni 2026

1. Problemstellung & Motivation

Eine Lebensmittelmarke, die ihre Produkte über Amazon verkauft, sammelt mit der Zeit hunderttausende Kundenbewertungen. Unser Datensatz (Amazon Fine Food Reviews) enthält rund 568.000 davon, und so viele Bewertungen liest niemand. Der Blick auf den Durchschnitt hilft auch nicht: Wer eine Bewertung abgibt, ist meistens zufrieden, also fällt der Schnitt fast immer gut aus. Eine Gesamtnote von „4,2 Sternen“ sagt einem Produktmanager nichts darüber, welches seiner Produkte gerade Ärger macht. Was ihn interessiert, ist die Abweichung: einzelne Produkte mit auffällig vielen negativen Bewertungen, oder ein Produkt, dessen Bewertungen gerade kippen, und falls eins auffällt die Frage, woran es liegt. Solche Fragen kann man von Hand beantworten, solange es um ein paar hundert Bewertungen geht, bei einer halben Million nicht mehr. Die automatisierte Sentiment-Analyse ordnet jede Bewertung als positiv, neutral oder negativ ein und macht die Textmenge auswertbar.

Warum NLP, wenn es Sterne gibt?

Der Datensatz bringt zu jeder Review schon eine Sternebewertung mit, also stellt sich die Frage, warum man den Text überhaupt analysiert, wenn die Wertung als Zahl vorliegt. Dafür gibt es zwei Gründe. Erstens existieren die Sterne nur auf der Verkaufsplattform: Support-Mails oder Social-Media-Kommentare kommen ohne Sterne, da steht ausschließlich Text, und auf diesen Kanälen ist die Sentiment-Analyse das einzige Signal, das man hat. Unsere Pipeline ist als Prototyp für solche Kanäle gedacht; die Amazon-Reviews verwenden wir, weil sie über die Sterne eine Vergleichsgröße liefern, an der sich die Klassifikation messen lässt. Zweitens ist ein Stern eine Zahl, während der Text die Begründung liefert. Aus einer 1-Stern-Bewertung geht nicht hervor, ob das Produkt schlecht schmeckte, die Verpackung undicht war oder die Lieferung zu spät kam, also drei Probleme mit drei verschiedenen Zuständigkeiten. Der Nutzen der Analyse liegt damit darin, die Gründe hinter einer Wertung sichtbar zu machen; die Klassifikation selbst ist dafür nur das Mittel.

Zwei Leitfragen

Aus dieser Motivation ergeben sich zwei Fragen, die das System beantworten soll und die zugleich seine beiden Verarbeitungswege bestimmen. Die erste richtet sich auf den vorhandenen Bestand: Welche Produkte fallen über die gesamte Bewertungshistorie hinweg durch einen hohen Negativ-Anteil auf, und das bei genügend Bewertungen, um

aussagekräftig zu sein? Kurz gesagt, wo brennt es? Diese Frage blickt auf alle bereits vorhandenen Reviews zurück, arbeitet mit großen Mengen und zielt auf Aggregation, weshalb sie der Batch-Lauf offline über den kompletten Datenbestand beantwortet. Die zweite Frage richtet sich auf neu eintreffenden Text, der keine Sternbewertung mitbringt, etwa einen Support-Kommentar oder einen Social-Media-Beitrag: Wie ist diese einzelne Äußerung einzuordnen, und das sofort? Hier gibt es keine historische Masse und keine Sterne als Stütze, sondern nur den einen Text, der gerade ankommt, was der Live-Pfad mit einer einzelnen Anfrage und sofortiger Antwort übernimmt.

Die beiden Fragen trennt dieselbe Linie, an der auch die Architektur entlangläuft. Auf der einen Seite steht die vorhandene Masse an Reviews, die Sterne mitbringen und rückblickend ausgewertet werden; auf der anderen der einzelne neue Text ohne Sterne, der unmittelbar eingeordnet werden muss. Deshalb verarbeitet das System Daten auf zwei getrennten Wegen, und das Dashboard zeigt beide nebeneinander.

2. Zielsetzung & Scope

Ziel der Arbeit ist ein lauffähiger Prototyp, der die beiden Leitfragen aus Abschnitt 1 technisch umsetzt und dabei die Konzepte aus der Vorlesung anwendet: einen Data Lake nach dem Medallion-Muster, eine geschichtete Batch-Verarbeitung und einen davon getrennten Pfad für Echtzeit-Inferenz. Konkret entsteht eine Pipeline, die die rohen Reviews aus einem Objektspeicher (MinIO) einliest, bereinigt, mit einem Sentiment-Verfahren klassifiziert und die Ergebnisse aggregiert. Eine API stellt diese Ergebnisse bereit, ein zweiter Dienst hostet ein neuronales Modell für die Live-Klassifikation einzelner Texte, und ein Dashboard macht beides sichtbar. Alle Komponenten laufen als getrennte Container über Docker Compose, sodass sich das System mit einem Befehl starten lässt.

Bewusst nicht Teil der Arbeit ist alles, was ein Produktivsystem ausmachen würde. Die Pipeline läuft nicht kontinuierlich, sondern wird manuell oder per Cronjob angestoßen; es gibt keine Authentifizierung, keine Hochverfügbarkeit und keine automatische Skalierung. Der Datenbestand ist statisch, ein echter Datenstrom mit fortlaufend eintreffenden Events wird nicht verarbeitet. Auch die Wahl von Apache Spark ist eher didaktisch motiviert als technisch notwendig, worauf Abschnitt 3 noch eingeht. Der Prototyp soll die Architektur und den Datenfluss demonstrieren, nicht den Betrieb unter Last.

3. Architektur

3.1 Überblick: zwei Verarbeitungswege

Das System trennt zwei Wege, die sich in ihren Anforderungen grundlegend unterscheiden. Der Batch-Weg verarbeitet den kompletten Bestand der rund 568.000 Reviews offline und in mehreren Stufen, vom Rohzustand bis zu fertig aggregierten Auswertungen. Der Live-Weg beantwortet einzelne Anfragen in Echtzeit: Ein Nutzer gibt einen Text ein, ein neuronales Modell klassifiziert ihn, die Antwort kommt sofort zurück. Beide Wege treffen sich erst im Frontend wieder, wo eine FastAPI als gemeinsame Schnittstelle die aggregierten Batch-Ergebnisse ausliefert und Anfragen an das Live-Modell weiterreicht.

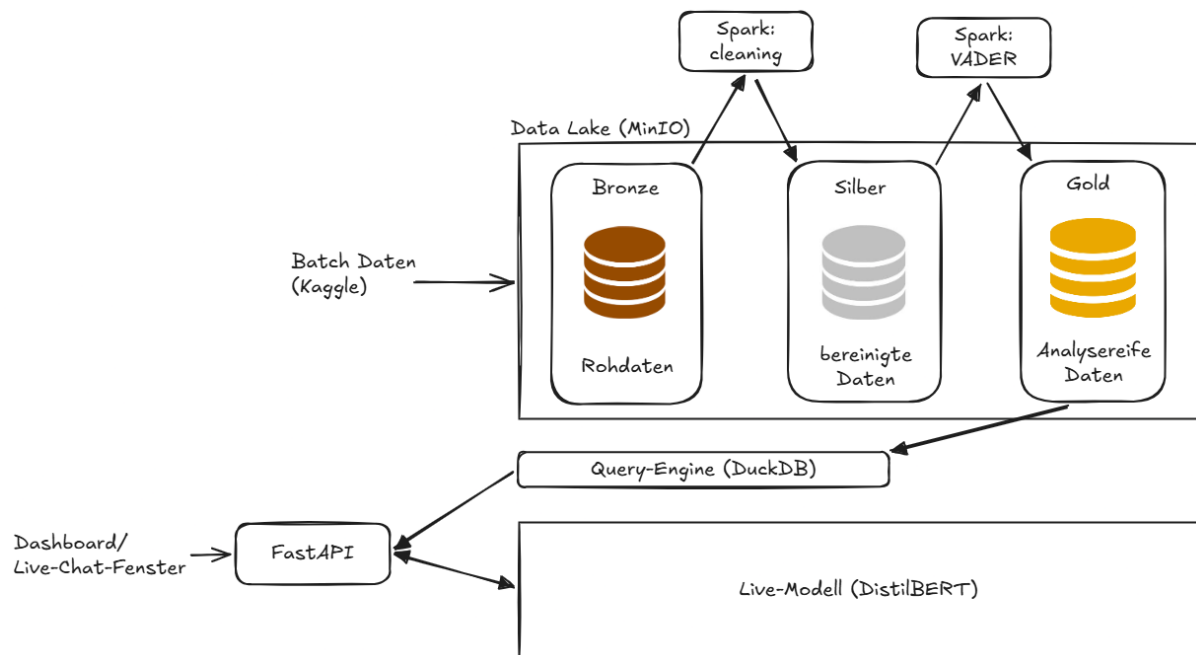


Abbildung 1: Systemarchitektur und Datenfluss des Prototyps (in Anlehnung an Databricks, o. J.).

Die Daten fließen im Batch-Weg von einer externen Quelle (Kaggle) in den Objektspeicher und durchlaufen dort drei Schichten, bevor eine Query-Engine die Ergebnisse für die API bereitstellt. Der Live-Weg hängt nicht an diesem Datenfluss, sondern ist ein eigenständiger Dienst, den die API bei Bedarf aufruft.

3.2 Container-Sicht

Jede Komponente läuft als eigener Container, orchestriert über Docker Compose. Damit lässt sich das gesamte System mit einem Befehl starten, und die Trennung der Dienste bildet die logische Trennung der Architektur ab.

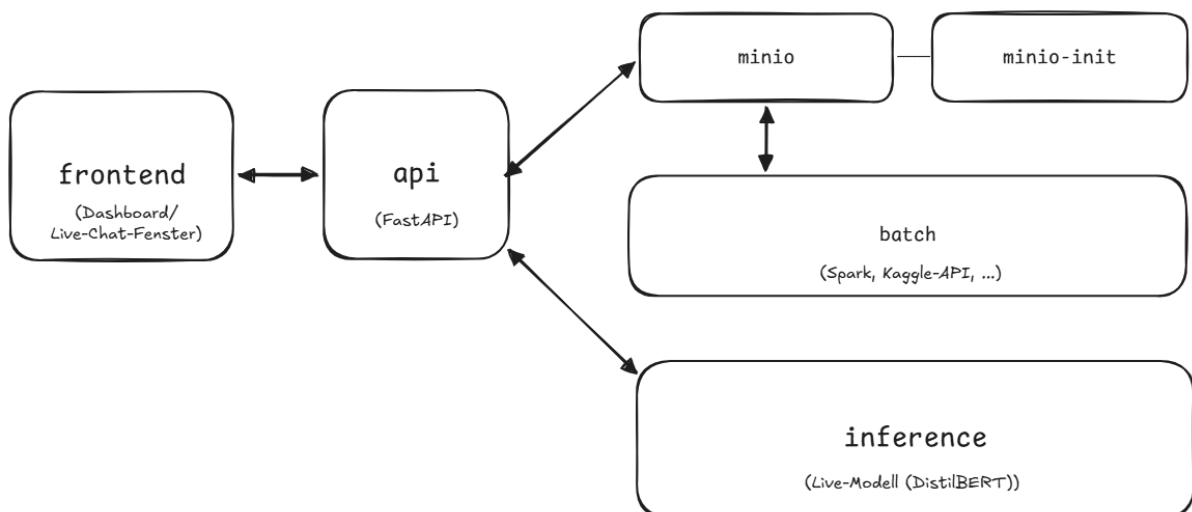


Abbildung 2: Container-Struktur und Dienst-Interaktionen im Docker-Compose-Netzwerk.

Im Zentrum steht MinIO als Speicher, auf den drei Dienste zugreifen: ein einmaliger Init-Container legt beim Start die Buckets an, der Batch-Orchestrator schreibt die verarbeiteten Daten hinein, und die API liest die fertigen Ergebnisse heraus. Der Batch-Orchestrator ist kein dauerhaft laufender Dienst, sondern ein Job, der seine Pipeline durchläuft und sich danach

beendet. Der Live-Inference-Container hält dagegen das neuronale Modell permanent geladen und wartet auf Anfragen. Dashboard und API laufen ebenfalls dauerhaft.

Im Repository spiegelt sich diese Trennung in vier Verzeichnissen wider (api, batch, frontend, inference), von denen jedes einen der Container definiert. Gestartet wird das System mit `docker compose up`; einzige Voraussetzung sind Kaggle-API-Credentials als Umgebungsvariablen, damit der Batch-Container die Rohdaten beziehen kann.

3.3 Data Lake und Medallion-Architektur

Der Objektspeicher MinIO dient als Data Lake und folgt dem Medallion-Muster mit drei Schichten. Die bronze-Schicht enthält die Rohdaten unverändert, so wie sie aus der Quelle kommen, also die CSV-Datei mit allen Originalspalten. Die silver-Schicht enthält den bereinigten Stand: reduziert auf die relevanten Spalten, von Duplikaten befreit und um leere Texte bereinigt. Die gold-Schicht enthält die fertig klassifizierten und aggregierten Ergebnisse, die das Dashboard direkt anzeigt. Jede Schicht baut auf der vorherigen auf, und die Trennung erlaubt es, eine Stufe neu zu berechnen, ohne die anderen anzufassen.

Während bronze die Daten im Originalformat (CSV) hält, speichern silver und gold als Parquet. Parquet ist ein spaltenorientiertes Format, das die Daten komprimiert ablegt und beim Lesen erlaubt, nur die benötigten Spalten zu laden. Für analytische Zugriffe, bei denen man oft nur einzelne Spalten über viele Zeilen aggregiert, ist das deutlich effizienter als zeilenorientiertes CSV. Außerdem behält Parquet die Datentypen bei, sodass nachgelagerte Werkzeuge wie Spark oder DuckDB die Daten ohne erneutes Parsen verarbeiten können.

3.4 Technologiewahl

MinIO stellt eine S3-kompatible Objektspeicher-API bereit und lässt sich lokal als Container betreiben. Damit dient es als Data Lake, ohne dass ein echter Cloud-Speicher nötig ist, und die S3-Schnittstelle sorgt dafür, dass die Anbindung später auf einen tatsächlichen Cloud-Speicher übertragbar bliebe.

Apache Spark übernimmt die Batch-Verarbeitung von bronze nach silver und gold. Spark läuft hier im local mode, also als ein einzelner Prozess statt als verteiltes Cluster. Bei der vorliegenden Datengröße reicht das vollständig aus; die verteilte Verarbeitung, für die Spark eigentlich gedacht ist, demonstrieren wir damit nur im Kleinen (siehe auch Abschnitt 7).

DuckDB liest die gold-Ergebnisse für die API aus, ist dabei aber keine weitere Speicherschicht, sondern eine eingebettete Query-Engine. Es läuft im Prozess der API, lädt keine Daten dauerhaft, sondern liest die Parquet-Dateien bei jeder Anfrage direkt aus MinIO und beantwortet darauf SQL-Abfragen. So bleibt die schwere Verarbeitung bei Spark, während DuckDB als schlanker Serving-Layer die Abfragen fürs Dashboard bedient.

FastAPI bildet die gemeinsame Schnittstelle nach außen. Sie liefert über DuckDB die aggregierten Batch-Ergebnisse aus und reicht Live-Anfragen an den Inference-Container weiter, hält selbst also keine Modell-Logik. Der Inference-Container hostet ein DistilBERT-Modell für die Sentiment-Klassifikation und ist als eigener Dienst getrennt, damit das Modell an genau einer Stelle lebt und die API schlank bleibt. Das Frontend ist mit React umgesetzt, weil es als eigenständige Anwendung im Browser läuft und die beiden Anwendungsfälle aus Abschnitt 1 übersichtlich nebeneinander darstellen kann.

3.5 Einordnung: Lambda-inspiriert, kein echtes Lambda

Die Architektur erinnert an die Lambda-Architektur aus der Vorlesung, deckt sich mit ihr aber nicht vollständig, und diese Unterscheidung ist uns wichtig. In der Lambda-Architektur laufen ein Batch-Layer für die genaue Verarbeitung historischer Daten und ein Speed-Layer für die sofortige Verarbeitung neu eintreffender Daten parallel, und ein Serving-Layer stellt die

kombinierten Ergebnisse bereit. Unser Batch-Weg mit dem Medallion-Muster entspricht dem Batch-Layer, und DuckDB zusammen mit dem Dashboard übernimmt die Rolle des Serving-Layers.

Der Live-Pfad ähnelt auf den ersten Blick einem Speed-Layer, ist aber keiner im eigentlichen Sinn. Ein Speed-Layer verarbeitet einen kontinuierlichen Datenstrom und hält eine fortlaufend aktualisierte Echtzeit-Sicht. Unser Live-Pfad dagegen arbeitet nach dem Request-Response-Prinzip: Er klassifiziert einen einzelnen, vom Nutzer übergebenen Text und schreibt das Ergebnis nirgends zurück. Es fließt also kein Stream, und die Live-Ergebnisse speisen auch keine gemeinsame Sicht zusammen mit den Batch-Daten. Korrekt beschrieben ist das System damit als Medallion-basierte Batch-Architektur mit einem zusätzlichen Online-Inferenz-Dienst. Die Bezeichnung „Lambda-inspiriert“ trifft es, eine echte Lambda- oder Kappa-Architektur läge nur vor, wenn tatsächlich ein Eventstrom verarbeitet würde (zur Kritik an der Lambda-Architektur und zur Kappa-Alternative vgl. Kreps 2014).

4. Datenverarbeitung (Pipeline)

4.1 Ingestion: Kaggle nach bronze

Am Anfang steht der Bezug der Rohdaten. Der Batch-Container lädt den Datensatz über die Kaggle-API herunter und legt die unveränderte CSV-Datei in der bronze-Schicht von MinIO ab. Dieser Schritt ist idempotent gestaltet: Bevor der Download startet, prüft der Container, ob die Datei bereits in bronze liegt, und überspringt den Bezug in diesem Fall. So lädt ein wiederholter Pipeline-Lauf nicht jedes Mal erneut rund 240 MB herunter, was die Entwicklung und das wiederholte Testen spürbar beschleunigt. Die bronze-Schicht bleibt damit der unangetastete Ausgangszustand, auf den sich alle weiteren Stufen zurückführen lassen.

4.2 Cleaning: bronze nach silver

Die Bereinigung übernimmt Spark. Beim Einlesen der CSV ist zu beachten, dass die Review-Texte selbst Kommas, Zeilenumbrüche und Anführungszeichen enthalten; ohne entsprechende Reader-Optionen (multiLine, escape) zerlegt Spark die Zeilen falsch und die Spalten verrutschen. Aus den Originaldaten behalten wir die für die Analyse relevanten Spalten (Id, ProductId, Score, Time, Text), verwerfen Zeilen mit leerem Text und entfernen Duplikate.

Die Deduplizierung ist hier kein Randdetail. Von den rund 568.000 Rohbewertungen bleiben nach dem Entfernen doppelter Texte etwa 394.000 übrig, also fallen rund 31 % als Duplikate weg. Der Grund liegt in der Natur des Datensatzes: Dieselbe Bewertung erscheint oft mehrfach, weil sie für verschiedene Produktvarianten desselben Artikels eingetragen wurde. Diese Beobachtung ist die praktische Entsprechung des Merkmals Veracity aus den fünf V's: Die Daten stammen aus einer realen Quelle und sind nicht ohne Weiteres vertrauenswürdig, sondern müssen erst auf Konsistenz geprüft und bereinigt werden, bevor eine Auswertung sinnvoll ist. Auch die übrigen V's lassen sich am Projekt festmachen. Volume steckt in den hunderttausenden Reviews, Variety in der Mischung aus strukturierten Feldern (Score, ProductId) und Freitext, Velocity in der Trennung von Batch-Lauf und sofort beantworteter Live-Anfrage und Value in der priorisierbaren Produktsicht, auf die Abschnitt 5.3 eingeht.

Ein zweites Qualitätsproblem sind HTML-Reste in den Texten. Viele Reviews enthalten Markup wie Zeilenumbruch-Tags oder eingebettete Links, die bei der Textauswertung stören: In einer Word Cloud tauchen sonst Fragmente wie „br“ oder „href“ unter den häufigsten Begriffen auf, und auch das Sentiment kann durch solche Reste verfälscht werden. Wir entfernen die Tags daher, bevor der Text visualisiert oder ausgewertet wird. Der bereinigte Stand wird als Parquet in die silver-Schicht geschrieben.

4.3 Sentiment: silver nach gold

Im Kern der Pipeline steht die Sentiment-Klassifikation. Für den Batch-Weg verwenden wir VADER, ein regel- und lexikonbasiertes Verfahren ohne Trainingsschritt. VADER stützt sich auf ein handgepflegtes Wörterbuch von rund 7.500 Begriffen, denen jeweils ein Valenzwert zwischen stark negativ und stark positiv zugeordnet ist. Über diese reinen Wortwerte legt das Verfahren mehrere Regeln, die menschliche Intensitätsmarker nachbilden: Ausrufezeichen und Großschreibung verstärken die Wertung, Gradadverbien wie „very“ verschieben sie, eine vorangestellte Verneinung dreht sie um, und ein „but“ gewichtet den Satzteil danach stärker. Die angepassten Einzelwerte werden zu einer Summe addiert und anschließend über eine Sättigungsfunktion auf einen Wert zwischen minus eins und plus eins normalisiert (den compound-Score). Aus diesem Score leiten wir das Label ab: ab plus 0,05 gilt eine Bewertung als positiv, ab minus 0,05 als negativ, dazwischen als neutral. Diese Schwellen sind die vom Verfahren empfohlenen Standardwerte.

Das Ergebnis fließt in die gold-Schicht. Dort liegen zum einen die einzeln klassifizierten Reviews mit ihrem compound-Wert und Label, zum anderen eine bereits aggregierte Verteilung über die drei Klassen, die das Dashboard ohne weitere Berechnung anzeigen kann.

4.4 DistilBERT-Modell: Live-Pfad

Den Live-Pfad bedient ein neuronales Modell, ein auf den Datensatz feinabgestimmtes DistilBERT mit drei Klassen (negativ, neutral, positiv). Es läuft im Inference-Container, hält das Modell dauerhaft geladen und beantwortet einzelne Klassifikationsanfragen. Anders als VADER erfasst es den Kontext eines Satzes über seine Aufmerksamkeitsmechanik, statt nur Wörter und lokale Muster zu zählen, was es bei Fällen wie Ironie oder sachlich formulierten Beschwerden zuverlässiger macht.

Für einen Vergleich der beiden Verfahren ist die Infrastruktur vorbereitet: Der Inference-Container stellt neben der Einzelabfrage einen Batch-Endpoint bereit, über den der Batch-Lauf Reviews ein zweites Mal mit dem HF-Modell scoren und das Ergebnis als zweite Spalte (sentiment_hf) neben das VADER-Label in die gold-Schicht schreiben könnte. Über den kompletten Bestand haben wir diesen Lauf nicht ausgeführt, weil das neuronale Scoring auf CPU für rund 394.000 Reviews zu lange dauert. Der quantitative Vergleich in Abschnitt 5 stützt sich deshalb auf VADER gegen die Sternbewertung; das HF-Modell prüfen wir qualitativ über den Live-Pfad.

5. Analyse & Ergebnisse

5.1 Verteilung der Sentiments

Die Auswertung des kompletten bereinigten Bestands ergibt eine stark unausgewogene Verteilung. Von den 393.579 klassifizierten Reviews ordnet VADER rund 88 % als positiv ein, etwa 10 % als negativ und nur knapp 2 % als neutral. Auf den ersten Blick wirkt das wie ein Fehler, ist aber ein erklärbarer Befund mit zwei Ursachen. Zum einen ist der Datensatz selbst positiv geprägt: Wer eine Lebensmittelbewertung schreibt, ist überwiegend zufrieden, sodass die wahre Verteilung tatsächlich Richtung positiv kippt. Zum anderen neigt VADER zusätzlich dazu, Texte als positiv einzustufen. Der besonders niedrige Neutral-Anteil ist ein Hinweis darauf: Wie in 4.3 beschrieben, summiert die Sättigungsfunktion viele schwach positive Einzelwerte auf und erreicht gerade bei längeren Texten schnell einen Wert nahe plus eins; echte Neutralität mit einem compound-Score um null entsteht so nur selten.

5.2 VADER gegen die Sternebewertung

Um diesen Eindruck zu prüfen, vergleichen wir das VADER-Label mit der Sternebewertung als Referenz. Dafür bilden wir die Sterne auf drei Klassen ab: ein bis zwei Sterne als negativ, drei als neutral, vier bis fünf als positiv. Die Sterne sind dabei nicht die absolute Wahrheit, aber die beste verfügbare unabhängige Vergleichsgröße. Über alle 393.579 Reviews stimmt VADER in 80,1 % der Fälle mit dieser Referenz überein. Diese Quote klingt zunächst ordentlich, ist aber an der Schiefeite der Daten zu messen: Ein triviales Verfahren, das jede Review pauschal als positiv einstuft, trüfe allein durch die 306.758 sternpositiven Reviews schon rund 78 %. VADER liegt damit nur etwa zwei Prozentpunkte über diesem Bezugswert, was die Gesamttrefferquote als Gütemaß weitgehend entwertet.

Wo das Verfahren danebenliegt, zeigt erst die Konfusionsmatrix. Von den 57.067 Reviews, die nach Sternen negativ sind, erkennt VADER nur 22.813 ebenfalls als negativ, also rund 40 %. Die mit Abstand größte Fehlerzelle umfasst 31.315 nach Sternen negative Reviews, die VADER ins Positive einsortiert; diese eine Fehlerklasse ist größer als die gesamte korrekt als negativ erkannte Gruppe. VADER schiebt damit mehr als die Hälfte der negativen Bewertungen ins Positive, genau die Verzerrung, die schon die Gesamtverteilung nahegelegt hat. Noch deutlicher ist die Schwäche bei der neutralen Klasse: Von den 29.754 nach Sternen neutralen Reviews ordnet VADER nur 1.130 auch als neutral ein, knapp 4 %, und verteilt den Rest auf positiv und negativ. Für ein Geschäftsziel, das gerade die negativen und kippenden Fälle finden soll, ist das die entscheidende Schwäche, denn die Mehrheit dieser Fälle übersieht das Verfahren. Die Ursache deckt sich mit den Beobachtungen an Einzelbeispielen: VADER bewertet Wörter und lokale Muster, erfasst aber keinen Satzkontext, sodass sachlich formulierte Beschwerden oder ironische Texte falsch eingeordnet werden.

An genau diesen Fällen ist der Mehrwert eines kontextsensitiven Modells zu erwarten. Das DistilBERT-Modell aus dem Live-Pfad gewichtet über seine Aufmerksamkeitsmechanik den Zusammenhang im Satz statt nur einzelne Wörter und sollte Ironie oder sachlich formulierte Beschwerden deshalb besser einordnen als ein Lexikonverfahren. Belegen lässt sich das auf diesem Korpus nicht quantitativ, weil der HF-Lauf über den gesamten Bestand auf der verfügbaren Hardware nicht praktikabel war (siehe Abschnitt 7). Die qualitative Prüfung am Live-Pfad stützt diese Erwartung, ein vergleichbarer Zahlenwert wie bei VADER fehlt dafür aber.

5.3 Auswertung aus Geschäftssicht

Für einen praktischen Nutzen reicht die Gesamtverteilung nicht aus, denn die Aussage „die meisten Reviews sind positiv“ trifft auf nahezu jeden Produktkatalog zu und führt zu keiner Entscheidung. Interessanter sind die Produkt-Triage, die das Dashboard bereitstellt, und eine zweite Beobachtung, die sich aus dem Vergleich von Text und Sternen ergibt.

Bei der Produkt-Triage zeigt ein Streudiagramm statt eines Gesamtdurchschnitts jedes Produkt als Punkt, mit der Zahl seiner Reviews auf der einen und dem Negativ-Anteil auf der anderen Achse. Produkte mit hohem Volumen und gleichzeitig hohem Negativ-Anteil heben sich oben rechts ab und sind die, um die man sich zuerst kümmern sollte. Ein Schwellenwert für die Mindestzahl an Reviews filtert dabei Einzelfälle heraus, bei denen eine einzelne negative Bewertung sonst einen Anteil von 100 % erzeugen würde.

Die zweite Beobachtung ergibt sich aus den Fällen, in denen Text und Sterne auseinanderfallen. Eine Bewertung mit fünf Sternen, deren Text aber negativ klingt, deutet oft auf ein Problem außerhalb des Produkts hin, etwa bei Versand oder Verpackung; der umgekehrte Fall kann auf Ironie oder eine falsch zugeordnete Bewertung hinweisen. Solche Diskrepanzen sind aus Geschäftssicht besonders aufschlussreich, weil sie genau die Signale enthalten, die eine rein sternebasierte Auswertung übersieht. Sie zeigen zugleich, dass der Vergleich zwischen Text-Sentiment und Sternebewertung über eine reine Gütemessung hinausgeht und einen inhaltlichen Hinweis liefert. Eine eigene Dashboard-Sicht dafür wäre ein

sinnvoller nächster Schritt, im aktuellen Prototyp ist diese Auswertung aber noch nicht umgesetzt.

6. Anwendung / Frontend

Das Dashboard ist die sichtbare Oberfläche des Systems und mit React umgesetzt. Sein Aufbau folgt direkt der Architektur: Weil das System Daten auf zwei getrennten Wegen verarbeitet, hat auch das Dashboard zwei Ansichten, eine für den Batch-Weg und eine für den Live-Weg. Beide Datenquellen erreicht das Frontend über die gemeinsame FastAPI.

Die Batch-Ansicht beantwortet die Frage „Wo brennt's?“. Im Zentrum steht das Streudiagramm der Produkt-Triage aus Abschnitt 5.3, ergänzt um einen Schieberegler für die Mindestzahl an Reviews und die Gesamtverteilung der Sentiments. Diese Auswertungen liest die API über DuckDB direkt aus der gold-Schicht. Da die Werte auf VADER beruhen, das negative Reviews unterschätzt, weist ein kurzer Hinweis darauf hin, dass die Anteile vorsichtig zu lesen sind und für einzelne Fälle die Live-Analyse genauer ist.

Bewusst nicht im Dashboard ist die Gegenüberstellung von VADER und Sternebewertung als Konfusionsmatrix. Sie misst die Güte des Verfahrens und beantwortet damit keine Geschäftsfrage, sondern dient der methodischen Bewertung. Diese Auswertung gehört in die Ergebnisdiskussion (Abschnitt 5), nicht in eine Oberfläche, die sich an einen Produktverantwortlichen richtet.

Die Live-Ansicht beantwortet die Frage „Neuer Text, sofort eingeordnet“. Sie besteht aus einem Eingabefeld, in das ein beliebiger Text eingegeben wird; auf Knopfdruck schickt das Frontend ihn über die API an den Inference-Container und zeigt das vom DistilBERT-Modell vergebene Label samt Konfidenz an. Damit lässt sich das Modell unmittelbar ausprobieren, auch an Texten, die nicht aus dem Datensatz stammen.

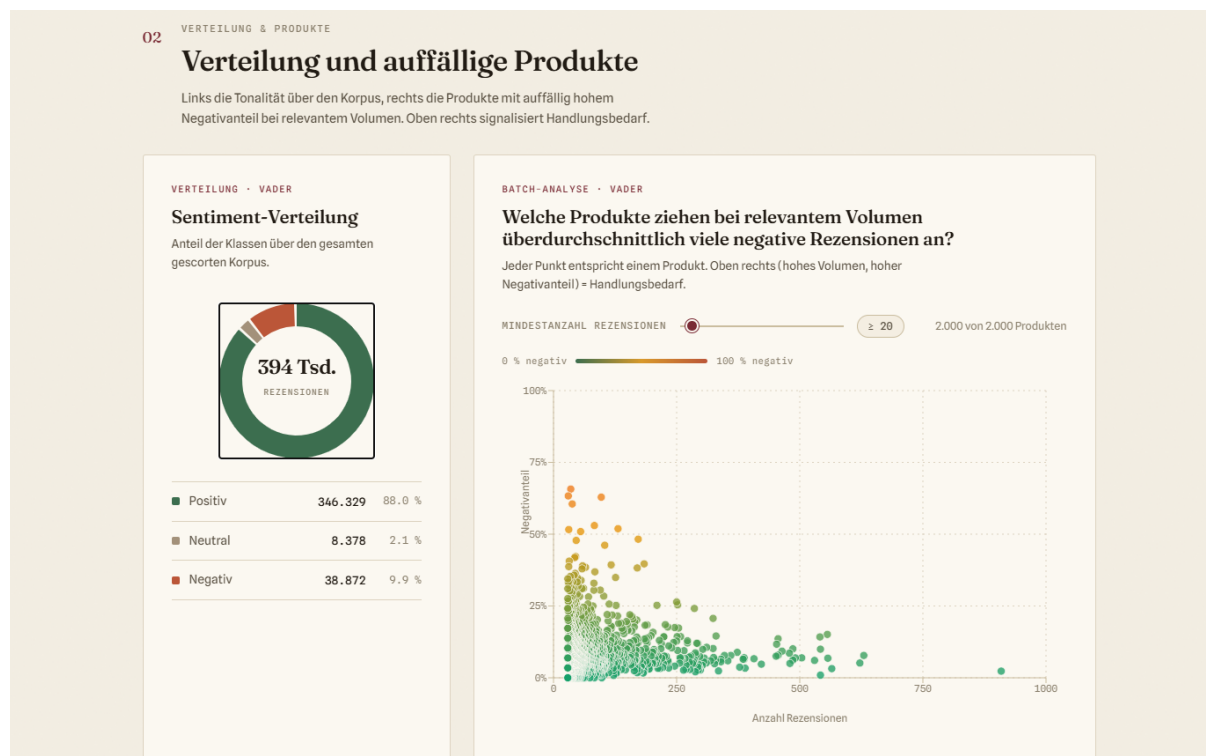


Abbildung 3: Batch-Ansicht mit Streudiagramm und Gesamtverteilung.

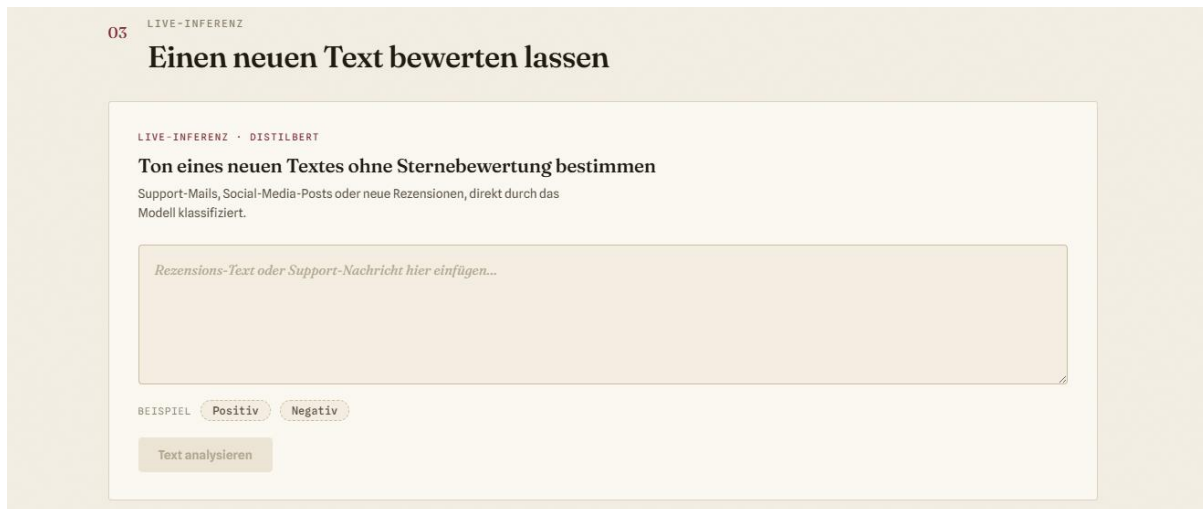


Abbildung 4: Live-Ansicht mit Eingabefeld und Klassifikationsergebnis.

Dass die beiden Ansichten nebeneinander und nicht hinter getrennten Reitern liegen, ist Absicht: Auf einen Blick wird erkennbar, dass hinter der einen Oberfläche zwei verschiedene Verarbeitungswege stecken.

7. Fazit & Limitationen

Der Prototyp setzt die beiden Leitfragen aus Abschnitt 1 vollständig und lauffähig um. Die Pipeline lädt die Rohdaten, bereinigt sie, klassifiziert das Sentiment und aggregiert die Ergebnisse über ein Medallion-Schema in MinIO; eine FastAPI stellt die Auswertungen über DuckDB bereit; ein getrennter Dienst beantwortet Live-Anfragen mit einem neuronalen Modell; und ein Dashboard macht beide Wege sichtbar. Das gesamte System startet als Verbund einzelner Container über Docker Compose. Inhaltlich zeigt die Auswertung einen konkreten Befund: VADER trifft die Sternebewertung in gut 80 % der Fälle, verschiebt dabei aber systematisch negative Bewertungen ins Positive und erkennt Neutralität kaum.

Bei den Grenzen der Arbeit ist Ehrlichkeit angebracht. Erstens ist der Einsatz von Apache Spark bei der vorliegenden Datengröße didaktisch motiviert und nicht technisch notwendig. Die rund 240 MB passen komplett in den Arbeitsspeicher, und ein einfacheres Werkzeug hätte denselben Zweck erfüllt. Spark demonstriert hier das skalierende Verarbeitungsmuster, nicht eine reale Notwendigkeit. Zweitens trägt die Sentiment-Auswertung im Batch die beschriebene Positiv-Verzerrung von VADER. Ein kontextsensitives Modell wie das DistilBERT aus dem Live-Pfad sollte diese Schwäche mildern, ein quantitativer Beleg über den gesamten Bestand war aber nicht möglich, weil die neuronale Klassifikation auf der verfügbaren CPU-Hardware für 394.000 Reviews zu lange läuft; der quantitative Vergleich bleibt deshalb auf VADER gegen die Sternebewertung beschränkt. Drittens ist das System bewusst ein Prototyp ohne Authentifizierung, Skalierung oder kontinuierlichen Betrieb, und es verarbeitet keinen echten Datenstrom, sondern arbeitet im Live-Pfad nach dem Request-Response-Prinzip; es ist damit Lambda-inspiriert, aber keine echte Lambda- oder Kappa-Architektur (siehe Abschnitt 3.5). Viertens entfernt die Deduplizierung Doppeltexte über alle Produkte hinweg, wodurch bei mehrfach verwendeten Bewertungen die Zuordnung zu einzelnen Produktvarianten verloren geht; für die aggregierte Auswertung ist das unkritisch, für eine produktgenaue Analyse wäre es zu verfeinern.

Insgesamt erfüllt der Prototyp seinen Zweck, das Zusammenspiel von Data Lake, geschichteter Batch-Verarbeitung und einem getrennten Inferenz-Dienst an einem realen Datensatz zu zeigen. Die offen benannten Grenzen markieren zugleich die sinnvollen nächsten Schritte: ein echter Eventstrom als Eingang, GPU-gestützte neuronale Klassifikation

im Batch, eine produktgenauere Behandlung der Duplikate und eine eigene Dashboard-Sicht für die Diskrepanzen zwischen Text und Sternebewertung (siehe Abschnitt 5.3).

8. Quellen

Databricks (o. J.): What is the Medallion Architecture? Online im Internet: <https://www.databricks.com/de/blog/what-is-medallion-architecture>. Abgerufen am: 01.06.2026.

Hugging Face (o. J.): DistilBERT. Transformers-Dokumentation. Online im Internet: https://huggingface.co/docs/transformers/model_doc/distilbert. Abgerufen am: 01.06.2026.

Hutto, C. J. (o. J.): vaderSentiment. GitHub-Repository mit Dokumentation des Verfahrens. Online im Internet: <https://github.com/cjhutto/vaderSentiment>. Abgerufen am: 01.06.2026.

Kaggle (o. J.): Amazon Fine Food Reviews (Stanford Network Analysis Project). Datensatzseite. Online im Internet: <https://www.kaggle.com/datasets/snap/amazon-fine-food-reviews>. Abgerufen am: 01.06.2026.

Kreps, J. (2014): Questioning the Lambda Architecture. O'Reilly Radar. Online im Internet: <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>. Abgerufen am: 01.06.2026.

Vorlesungsunterlagen Cloud Computing & Big Data, DHBW Lörrach (Handout Cloud Computing; Vorlesung Big Data Verarbeitung; Gegenüberstellung Lambda- und Kappa-Architektur).

Projekt-Dokumentationen der eingesetzten Werkzeuge, jeweils abgerufen am 01.06.2026: MinIO (min.io), Apache Spark (spark.apache.org), DuckDB (duckdb.org), FastAPI (fastapi.tiangolo.com), React (react.dev), Docker (docs.docker.com).

9. Kennzeichnung von KI-Inhalten

Im Rahmen der Arbeit wurden KI-Werkzeuge eingesetzt, im Wesentlichen Claude (Anthropic) sowie Claude Code. Der Einsatz erstreckte sich auf folgende Bereiche:

- Diskussion und kritische Prüfung des Architekturentwurfs sowie der Diagramme.
- Erzeugung von Code-Gerüsten und wiederkehrenden Bestandteilen (Docker-Compose-Konfiguration, Dockerfiles, Grundgerüst der Pipeline, API-Endpunkte, React-Komponenten).
- Unterstützung beim Verfassen und Überarbeiten von Teilen dieser Dokumentation.

Alle von KI-Werkzeugen erzeugten Inhalte wurden geprüft, angepasst und in den Gesamtkontext der Arbeit eingeordnet. Die fachlichen Entscheidungen, der Aufbau der Architektur und die Auswertung der Ergebnisse liegen bei den Autoren.